The True Cost of an Action

Cost per model call is the vendor’s number. Cost per accepted, completed action is yours.

Version 1.0 · October 2026 · Nitin Gupta

AI business cases are usually built on the cheapest number available: the price of a model call or a token. That number is real, and it is almost never the number that decides whether an AI system is worth running. Once human review, retries, rework, reversals, monitoring, evaluation, and shared platform costs are counted, the honest cost of one useful outcome can be many times the figure the business case was approved on.

This worksheet calculates that honest number. It needs no software, only a month of operating data and a willingness to count everything.

It produces four figures:

1. **True cost per accepted action:** what one useful outcome actually costs.
2. **Action yield:** what share of attempted actions become useful outcomes.
3. **Margin per accepted action:** the true cost compared with doing the same work without AI.
4. **Break-even review rate:** how much human review the system can need before it stops paying for itself.

The unit: an accepted, completed action

Everything is measured per **accepted, completed action**: an action the system attempted that executed, was accepted by the person or process receiving it, and was not reversed or reworked within a defined window (for example, 14 days).

Write the definition down for your system before you count anything, including the acceptance test and the window. Then count:

Count	Definition	This period
Attempted actions	Every action the system started, including ones that failed or were rejected	
Accepted, completed actions	Attempted actions that passed the acceptance test and were not reversed or reworked in the window	
Reviewed actions	Accepted actions that required a person to review, approve, or correct before release	

Action yield = accepted, completed actions divided by attempted actions.

A yield well below 100 percent is not a failure on its own. It is the number that tells you how much of the work you are paying for never becomes value.

The six cost layers

Count every cost in the period for this system, in six layers. If a cost is shared with other systems, allocate it by a written rule and note the rule.

Layer	What it includes	This period
Run	Model inference, tokens, compute, hosting	
Reach	Tool and API calls, retrieval, search, data access fees, integration and middleware costs	
Review	Time people spend reviewing, approving, or correcting outputs, at fully loaded labor cost	
Repair	Retries, failed actions, reversals, rework, exception handling, and customer remediation	
Assure	Evaluation runs, regression testing, monitoring, logging and trace storage, red-teaming, audit time	
Share	Allocated platform licenses, shared infrastructure, support, and the system’s share of the team that runs it	
Total		

Run is usually the smallest layer. Review and Repair are usually the largest. If your total is mostly Run, check whether the other layers were actually counted.

The four figures

1. True cost per accepted action

> Total of the six layers ÷ accepted, completed actions

2. Action yield

> Accepted, completed actions ÷ attempted actions

3. Margin per accepted action

> Baseline cost per action − true cost per accepted action

The **baseline cost per action** is what the same accepted outcome costs today without the AI system, using the current process, at fully loaded cost. Measure it, do not estimate it from a slide. If there is no current process, use the cheapest credible alternative.

A positive margin means the system is cheaper than the alternative. It does not mean the system is worth running; value and risk still matter. A negative margin means the system is costing more than the work it replaces, whatever its accuracy.

4. Break-even review rate

Human review is the cost that grows fastest when quality slips, and it is the first thing a business case underestimates. The break-even review rate shows how much review the system can absorb before it costs more than the baseline.

> Non-review cost per accepted action = (Total − Review layer) ÷ accepted, completed actions
>
> Review cost per reviewed action = Review layer ÷ reviewed actions
>
> **Break-even review rate = (Baseline cost − non-review cost per accepted action) ÷ review cost per reviewed action**

Compare it with the **current review rate** (reviewed actions ÷ accepted, completed actions).

- If the current rate is well below break-even, the system has room to absorb a quality dip.
- If the current rate is close to break-even, a small drop in quality will make the system uneconomic. Treat this as a risk, not just a cost issue.
- If the break-even rate is zero or negative, the system costs more than the baseline even with no human review at all. No improvement in review efficiency will fix it.

Worked example

Illustrative figures for one month, for a system that drafts and sends routine customer account adjustments. These are not benchmarks.

Count	Value
Attempted actions	10,000
Accepted, completed actions	8,600
Reviewed actions	2,000
Model calls	40,000

Layer	Cost
Run	$4,000
Reach	$2,500
Review (2,000 reviews at $5.00)	$10,000
Repair	$3,200
Assure	$4,500
Share	$3,000
Total	**$27,200**

Figure	Calculation	Result
Cost per model call (the vendor's number)	$4,000 ÷ 40,000	$0.10
True cost per accepted action	$27,200 ÷ 8,600	**$3.16**
Action yield	8,600 ÷ 10,000	86%
Baseline cost per action (current manual process)	measured	$6.00
Margin per accepted action	$6.00 − $3.16	$2.84
Non-review cost per accepted action	($27,200 − $10,000) ÷ 8,600	$2.00
Review cost per reviewed action	$10,000 ÷ 2,000	$5.00
Break-even review rate	($6.00 − $2.00) ÷ $5.00	**80%**
Current review rate	2,000 ÷ 8,600	23%

The business case for this system was approved on the ten-cent figure. The true cost is about thirty times higher. The system is still worth running, with a margin of $2.84 per useful outcome and a comfortable gap between its current review rate and its break-even point. The value of the worksheet is that the enterprise now knows that for a fact instead of hoping it. It also knows how much quality the system can lose before that stops being true.

Pilot versus production

Pilot economics almost always flatter the system. Before approving a move to production, recalculate at expected production volume:

Question	Why it changes the answer
Which layers grow with volume?	Run, Reach, Review, and Repair usually scale with every action.
Which layers were missing from the pilot?	Assure and Share are often absorbed by a central team during a pilot and arrive later.
Will the review rate hold?	Pilots use enthusiastic, expert reviewers. Production uses everyone.
Will yield hold?	Production brings cases the pilot never saw.
Is the baseline still right?	The manual process the pilot compared against may already be improving.

If the margin at production volume is negative, or the current review rate at production volume would sit close to break-even, the system is not ready to scale, however strong its accuracy looked in the pilot.

How to use it

- **Calculate it monthly** for any system holding authority above Observe, and bring all four figures to the scheduled authority review.
- **Set a cost ceiling** per accepted action when funding is approved, and treat crossing it as a review trigger.
- **Watch the break-even gap, not just the cost.** The distance between the current review rate and the break-even rate is an early warning that quality and economics are about to fail together.

Lineage

Activity-based costing and the unit economics practice of the FinOps community both inform this worksheet. What it adds for AI specifically is the accepted, completed action as the unit of cost, a six-layer structure that makes human review and repair impossible to leave out, and the break-even review rate, which ties the economics of a system directly to its quality.

Part of the Enterprise AI Decision Toolkit, a set of open instruments for enterprise AI governance based on *The Enterprise AI Decision System: How AI Leaders Turn Capability into Authorized Enterprise Action* by Nitin Gupta. Latest version: github.com/ngupta1137/enterprise-ai-decision-toolkit. Licensed under MIT.